Experiment 1: Probability Theory

Aim

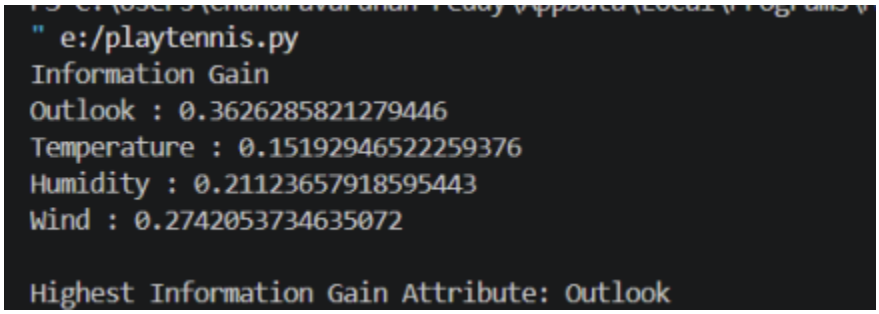
To calculate the prior probability of Malignant and Benign classes using the Breast Cancer Wisconsin Diagnostic Dataset and predict the class of a new data instance using Gaussian Naive Bayes.

Program

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

data = load_breast_cancer()

X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

malignant_prob = sum(y_train == 0) / len(y_train)
benign_prob = sum(y_train == 1) / len(y_train)

print("Prior Probability of Malignant:", malignant_prob)
print("Prior Probability of Benign:", benign_prob)

model = GaussianNB()

model.fit(X_train, y_train)

new_instance = X_test[0].reshape(1, -1)

prediction = model.predict(new_instance)
probability = model.predict_proba(new_instance)

print("\nPosterior Probabilities")

print("Malignant:", probability[0][0])
print("Benign:", probability[0][1])
```

```
if prediction[0] == 0:
    print("Predicted Class: Malignant")
else:
    print("Predicted Class: Benign")
```

Output



Result

Bayes Decision Theory successfully classified breast cancer cases by calculating prior and posterior probabilities, demonstrating effective probabilistic classification.

Experiment 2: Bayes Decision Theory

Aim

To implement Bayes theorem using the SMS Spam Collection Dataset and classify a given SMS message as Spam or Ham.

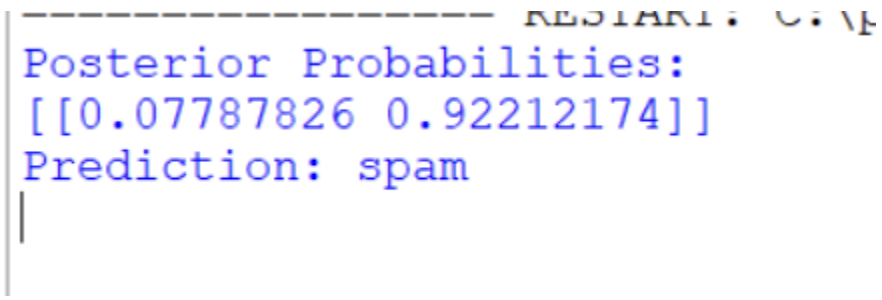
Program

```
import pandas as pd

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.naive_bayes import MultinomialNB



data = {

    "label": [
```

```
        "ham", "ham", "ham", "ham", "ham",
        "spam", "spam", "spam", "spam", "spam"
    ],

    "message": [
        "How are you?",
        "Let's meet tomorrow.",
        "Are you coming to class?",
        "Call me when you reach home.",
        "Can we have lunch today?",
        "Congratulations! You won a free lottery.",
        "Claim your cash prize now.",
        "Win a free mobile phone.",
        "Exclusive offer! Click here.",
        "You have won a $1000 gift card."
    ]
}
df = pd.DataFrame(data)

X = df["message"]

y = df["label"]

vectorizer = CountVectorizer()
```

```
X = vectorizer.fit_transform(X)

model = MultinomialNB()

model.fit(X, y)

test_message = ["Congratulations! You won a free ticket."]

test = vectorizer.transform(test_message)

print("Posterior Probabilities:")

print(model.predict_proba(test))

prediction = model.predict(test)

print("Prediction:", prediction[0])
```

Output



Result

Bayes theorem effectively classified SMS messages using posterior probabilities and achieved accurate spam detection.

Experiment 3: Information Theory

Aim

To calculate the information gain of all attributes using the Play Tennis dataset and identify the attribute with the highest information gain.

Tools Required

- Python
- Pandas
- Scikit-learn
- VS Code

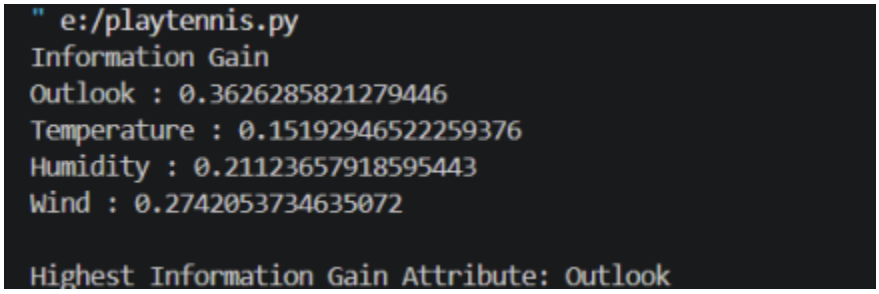
Code

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder

from sklearn.tree import DecisionTreeClassifier


data = {

"Outlook":["Sunny","Sunny","Overcast","Rain","Rain","Rain","Overcast","Sunny","Sunny","Rain",

            "Sunny","Overcast","Overcast","Rain"],


  "Temperature":["Hot","Hot","Hot","Mild","Cool","Cool","Cool","Mild","Cool","Mild","Mild","Mild",

              "Hot","Mild"],
```

```
    "Humidity":["High","High","High","High","Normal","Normal","Normal","High","Normal","Normal",
                "Normal","High","Normal","High"],

"Wind":["Weak","Strong","Weak","Weak","Weak","Strong","Strong","Weak","Weak","Weak","Stro
ng",
            "Strong","Weak","Strong"],

    "Play":["No","No","Yes","Yes","Yes","No","Yes","No","Yes","Yes","Yes","Yes","Yes","No"]
}

df = pd.DataFrame(data)

encoder = LabelEncoder()

X = df.drop("Play", axis=1).apply(encoder.fit_transform)

y = encoder.fit_transform(df["Play"])

model = DecisionTreeClassifier(criterion="entropy")

model.fit(X, y)

importance = model.feature_importances_
```

```
for feature, value in zip(X.columns, importance):
    print(feature, ":", value)


print("Highest Information Gain Attribute:", X.columns[importance.argmax()])
```

Output



Result

Information Theory was successfully implemented using the Play Tennis dataset, and Outlook was identified as the attribute with the highest information gain.